Software Engineering
Project Report

SENTINEL VULNERABILITY SCANNER

Prepared by:-

Anindya Bhattacharjee 23/49046
Harpreet Kaur 23/49013

BSc(Hons) Computer Science,Dyal Singh College

Table of Contents

1. **Acknowledgement**

2. **Certificate**

3. **Chapter 1:** Problem Statement & Feasibility Study

4. **Chapter 2:** Software Requirements Specification

5. **Chapter 3:** Project Management

6. **Chapter 4:** Design and Implementation

7. **Chapter 5:** Testing

8. **References**

Acknowledgment

We would like to express our sincere gratitude to Dr. Anita Goel and Dr. Sheetal Rajpal, Department of Computer Science, Dyal Singh College, for their invaluable guidance, motivation, and constant encouragement throughout the development of our project *"Sentinel Vulnerability Scanner."* Their expert insights and constructive feedback have played a crucial role in shaping the direction and quality of this work.

We also extend our heartfelt thanks to all the faculty members of the Department of Computer Science for providing a strong academic foundation and a supportive environment that enabled us to explore and implement this project effectively.

We are grateful to Dyal Singh College for offering us the necessary resources, infrastructure, and learning opportunities that contributed significantly to the successful completion of this project.

We would also like to acknowledge our classmates and peers for their cooperation, insightful discussions, and continuous support during the various stages of research, design, and implementation.

Finally, we express our deep appreciation to our families for their unwavering support, encouragement, and understanding throughout this journey.

Certificate

This is to certify that the project entitled

“Sentinel Vulnerability Scanner”

submitted by

Anindya Bhattacharjee — **23/49046**

Harpreet Kaur — **23/49013**

students of
B.Sc. (Hons.) Computer Science, Semester V,
Dyal Singh College, University of Delhi,

has been completed under our supervision for the Academic Year 2024–2025.

This report is submitted in partial fulfillment of the requirements for the degree of
Bachelor of Science (Hons.) in Computer Science.

Dr. Anita Goel

Teacher- in-charge

Dr. Sheetal Rajpal

Teacher- in-charge

Date:- 17th November 2025

Chapter 1: Problem Statement and Feasibility Study

1.1 Problem Statement

In today's digital ecosystem, organizations increasingly rely on interconnected systems, web applications, and networked services. While these systems improve productivity, they also expand the attack surface, exposing organizations to a wide range of cyber threats such as unauthorized access, malware infiltration, misconfigurations, and unpatched vulnerabilities.
Traditional manual scanning methods or outdated tools often fail to provide real-time, actionable insights, leaving critical weaknesses undiscovered.

Security analysts and testers require a centralized, automated, and intelligent vulnerability scanning system that can identify open ports, detect running services, enumerate software versions, and highlight potential risks effectively. Additionally, decision-makers and managers need clear, visually structured reports for auditing, compliance, and further action.

However, there is a gap in existing solutions:

1. Many tools are either too complex for small teams or lack AI-assisted insights.

2. Some open-source scanners require extensive configuration.

3. Report generation is often manual and time-consuming.

4. Existing tools rarely combine vulnerability scanning + AI analysis + downloadable reporting in a single lightweight platform.

Sentinel Vulnerability Scanner aims to solve these gaps by providing:

- Automated network scanning (open ports, service detection, version enumeration)
- AI-driven risk interpretation and recommendations
- Clean, downloadable reports for audits
- Simple web interface accessible from a browser

This system empowers organizations to identify, analyze, and respond to vulnerabilities faster and more efficiently, reducing exposure and strengthening network security.

1.2 Feasibility Study

A feasibility study evaluates whether the proposed Sentinel Vulnerability Scanner can be developed and deployed effectively in terms of technical, economic, operational, and schedule-related constraints.

1.2.1 Technical Feasibility

The project is technically feasible due to the following factors:

- **Backend Technology:**
  FastAPI supports fast request handling, integrations, and asynchronous operations—ideal for network scanning and AI usage.
- **Scanning Engine:**
  The system uses Nmap via `python-nmap`, which is widely supported on Linux, Windows, and macOS.

- **Frontend Technology:**
  React.js provides an interactive UI for scanning input, chat assistance, and report viewing.

- **AI Integration:**
  Mistral API is used for intelligent suggestions and risk summaries, requiring only API access and lightweight model configuration.

- **Database & Storage:**
  Mongo DB allows easy saving of scans and reports.

Since all tools and libraries are stable and well-documented, the system is **technically viable**.

1.2.2 Economic Feasibility

- Most technologies used (React, FastAPI, Python, Nmap) are **open source**.

- Deployment can be done on low-cost cloud servers or even local systems.

- No licensing fees required for scanning or AI usage (if using local API or open-source models).

- The project reduces manual effort and operational costs by offering automated scanning and reporting.

Thus, the system is **cost-effective** and financially feasible for students, startups, and mid-scale organizations.

1.2.3 Operational Feasibility

The system is easy to use and integrates smoothly with existing workflows:

- Analysts and testers can run scans via a simple UI rather than command-line tools.

- AI assistance reduces the need for deep domain knowledge during analysis.

- Managers can view/download reports without performing scans.

- The interface is designed to be intuitive — minimal training required.

This ensures the solution is **operationally feasible**.

1.2.4 Schedule Feasibility

Given the project structure and available technology stack, development tasks—user interface, backend APIs, scanning module, AI integration, database, and reporting—can be completed within the academic project timeline (8–10 weeks).

The use of open-source libraries and clear modular division further supports timely completion, making the project **schedule-feasible**.

Conclusion of Feasibility Study

The Sentinel Vulnerability Scanner is **fully feasible** across technical, economic, operational, and schedule dimensions. With available open-source tools, well-defined requirements, and structured implementation steps, the system can be successfully developed and deployed to provide meaningful security insights and automated reporting.

Chapter 2: Software Requirement Specification

Introduction

2.1.1 Purpose: This Software Requirements Specification (SRS) document defines the requirements for the Sentinel Vulnerability Scanner (Version 1.0). Sentinel is a cybersecurity tool designed to detect, analyze, and report vulnerabilities in computer systems, networks, and applications. The purpose of this document is to specify the functional and non-functional requirements of the system to ensure its effective design, development, and deployment.

2.1.2 Document Conventions: This document follows standard conventions, where:

● Bold is used for section headings.

Priorities for requirements are inherited by detailed requirements unless stated otherwise.

2.1.3 Intended Audience and Reading Suggestions: This document is intended for developers, project managers, testers, end users, and documentation writers of the *Sentinel Vulnerability Scanner (v1.0)*.

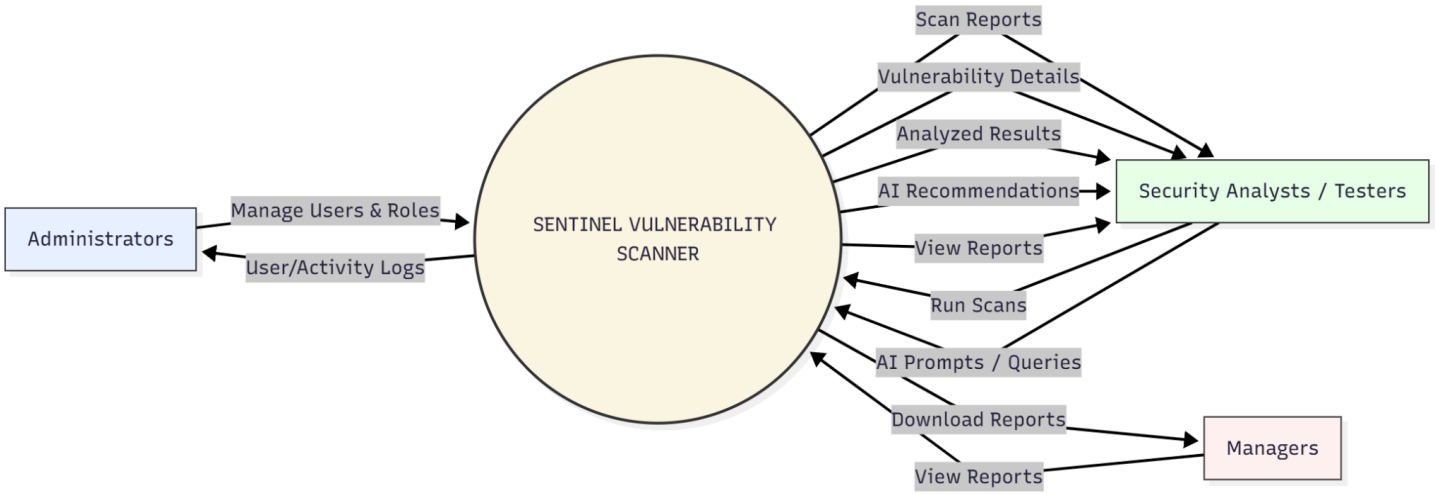
- Developers/Testers ↠ focus on Functional & System Requirements.
- End Users ↠ review System Features and Use Cases.

Readers should begin with the Introduction and Overview, then proceed to the sections most relevant to their role.

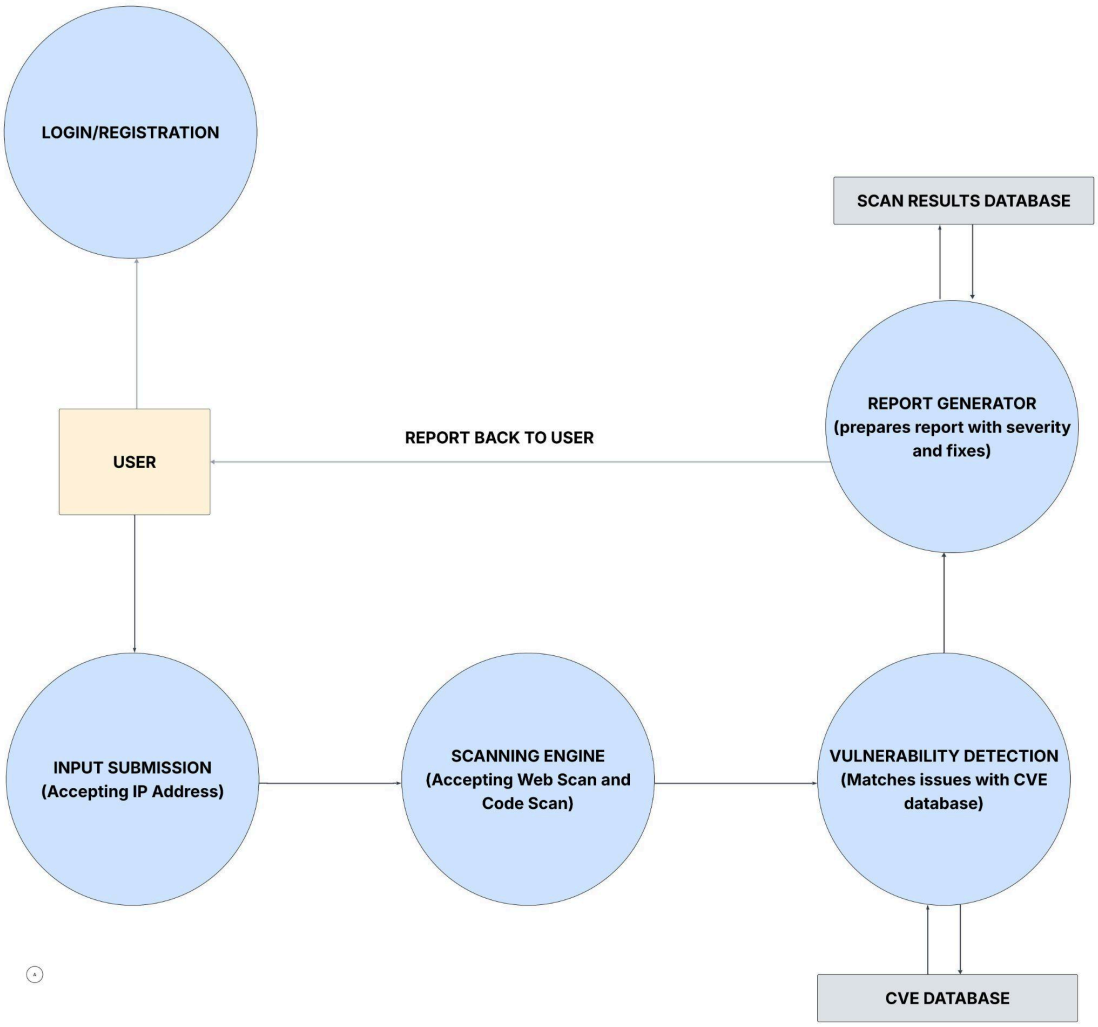
2.1.4 Product Scope: The Sentinel Vulnerability Scanner (v1.0) is a cybersecurity tool designed to identify, analyze, and report security vulnerabilities in web applications. Its primary goal is to help organizations and users strengthen their security posture by detecting weaknesses before they can be exploited.

Overall Description

2.2.1 Product Perspective: The Sentinel Vulnerability Scanner (v1.0) is a new, self-contained software product developed to provide automated vulnerability scanning and security analysis. It is not a follow-on member of an existing product family but has been designed from the ground up to integrate vulnerability detection with AI-powered assistance for remediation.

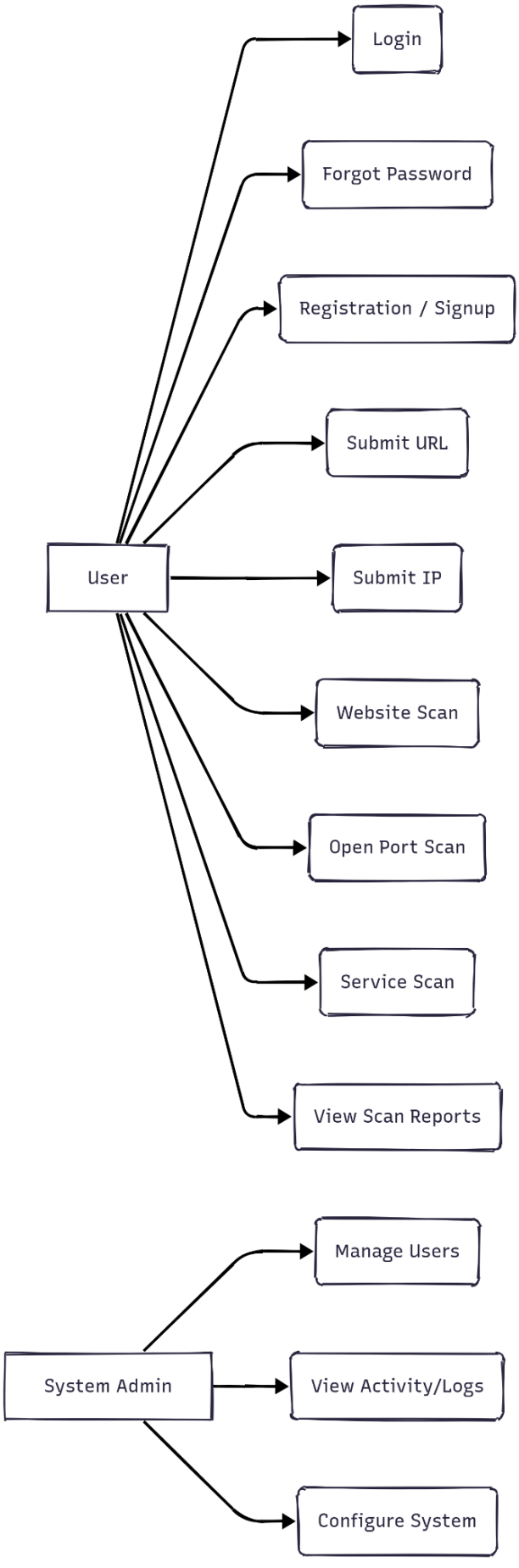


Context Diagram


LOGIN/REGISTRATION
SCAN RESULTS DATABASE
REPORT GENERATOR
(prepares report with severity and fixes)
USER
REPORT BACK TO USER
INPUT SUBMISSION
(Accepting IP Address)
SCANNING ENGINE
(Accepting Web Scan and Code Scan)
VULNERABILITY DETECTION
(Matches issues with CVE database)
CVE DATABASE

DFD LEVEL 1

2.2.2 Product Functions:



2.2.3 User Classes and Characteristics: The Sentinel Vulnerability Scanner will be used by different categories of users, each with distinct roles, privileges, and technical expertise:

1. **Administrators**
    - Role: Manage user accounts, assign roles, configure system settings.
    - Expertise: High technical knowledge of cybersecurity and system management.
    - Frequency of Use: Regular.
    - Privileges: Full access, including adding/removing users and viewing all reports.
    - Importance: Most critical user class, as they control and maintain the system.
2. **Security Analysts / Testers**
    - Role: Perform vulnerability scans, analyze results, apply remediation steps.
    - Expertise: Moderate to high cybersecurity knowledge.
    - Frequency of Use: Frequent.
    - Privileges: Access to scanning and reporting modules, but limited administrative controls.
    - Importance: High priority, as they are the main operational users.

3. **Managers**

    - Role: View reports, receive AI-generated recommendations, and act upon them.
    - Expertise: Low to moderate technical knowledge.
    - Frequency of Use: Occasional.

2.2.4 Operating Environment: The Sentinel Vulnerability Scanner (v1.0) will operate in a standard computing environment suitable for cybersecurity tools. The following specifications describe the expected environment:

- Hardware Platform:
    - Minimum: Intel Core i3 (6th Gen) or equivalent, 8 GB RAM, 1 TB HDD.
    - Recommended: Intel Core i5 or above, 16 GB RAM, 512 GB SSD.

- Operating System:
    - Primary: Linux distributions (e.g., Ubuntu 20.04 LTS, Parrot OS, Kali Linux).
    - Secondary: Windows 10/11 (for non-Linux users).

- Software Dependencies:
    - Python 3.9 or above (for core functionalities).
    - Database: MongoDB (for user management and scan result storage).
    - Web Framework: React/FastAPI (for account management and reporting interface).
    - AI Integration: Compatible with open-source LLM APIs (e.g., Mistral model).

- Network Requirements:

    - Active internet connection for updates, AI assistance, and external vulnerability database access (e.g., CVE feeds).
    - Local area network (LAN) support for scanning internal systems.

- Coexistence with Other Software:
    - Should not interfere with existing security tools (e.g., firewalls, intrusion detection systems).
    - Must run without conflict alongside common productivity applications.

This environment ensures the software can perform efficiently while being flexible for both academic and professional use.

External Interface Requirement

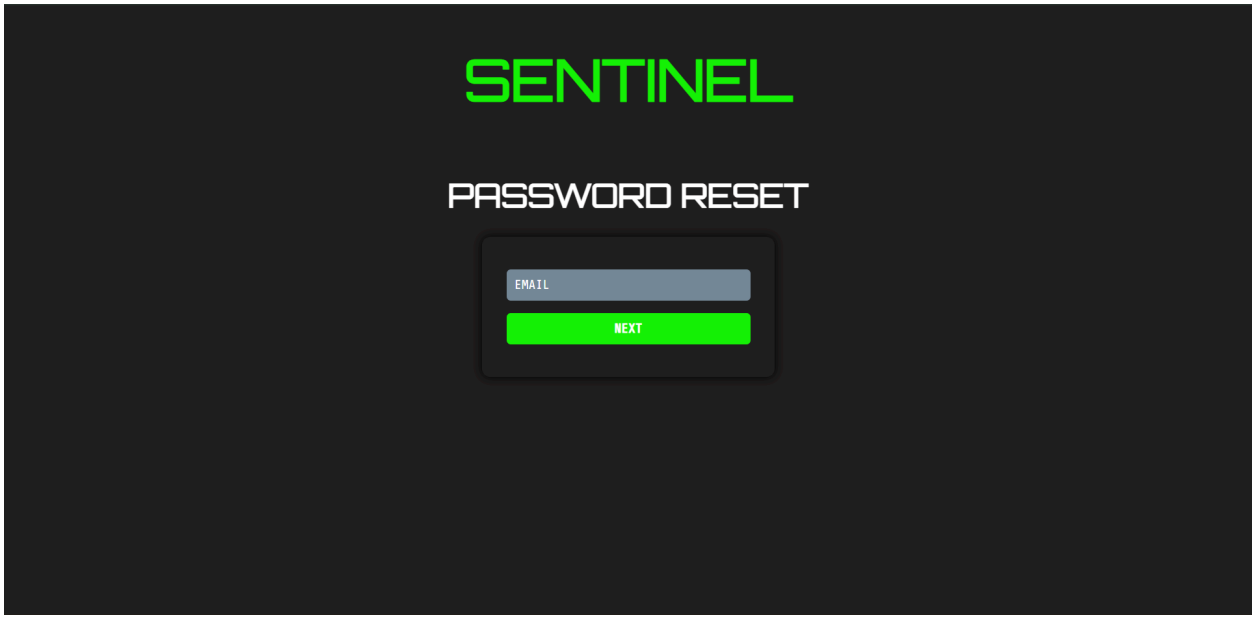
2.3.1 User Interfaces:



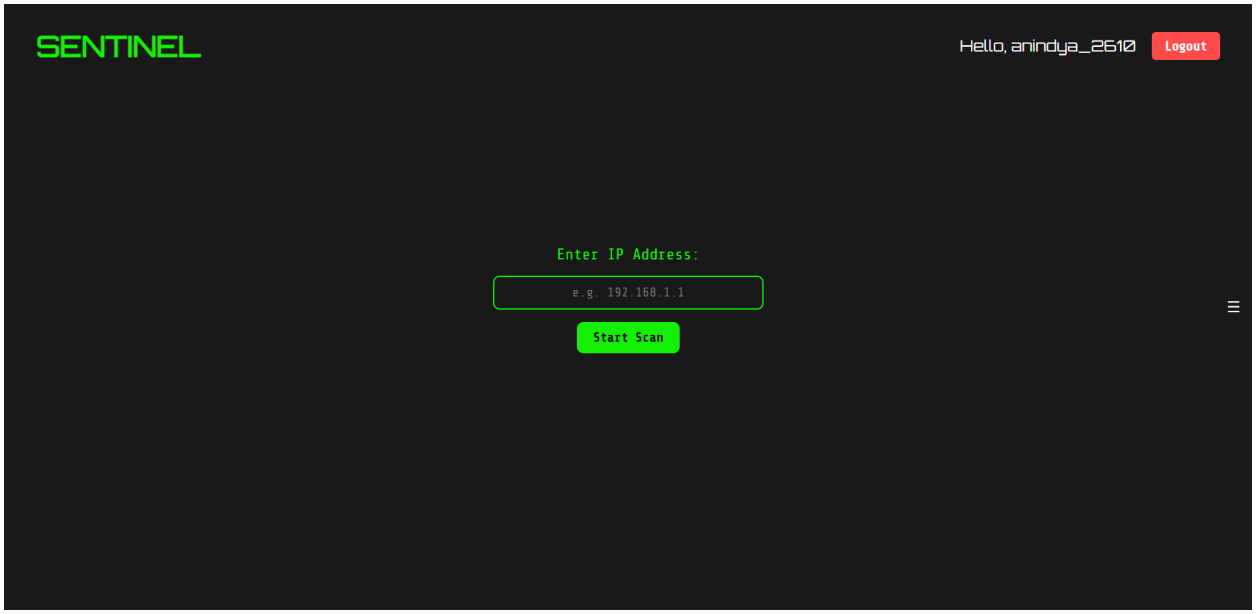
Login Page



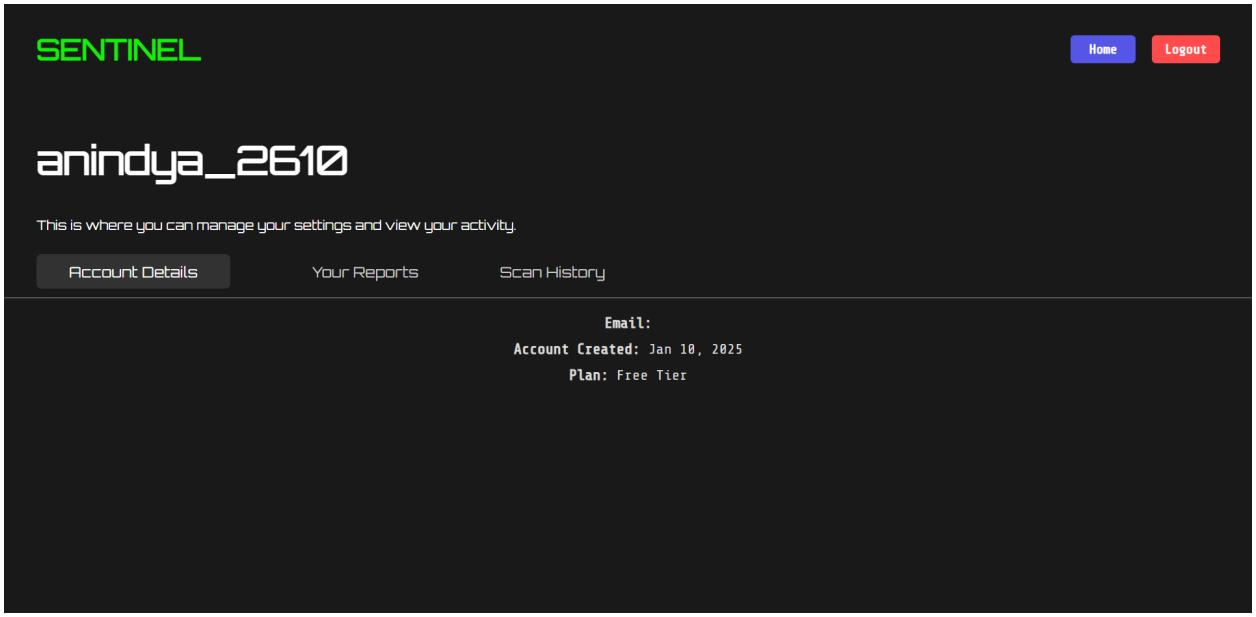
Signup Page



Password Reset Page



Scan Page



Profile Page



AI Page

2.3.2 Hardware Interfaces: The Sentinel Vulnerability Scanner (v1.0) interacts with standard computing hardware.

2.3.3 Software Interfaces: The system uses MongoDB for storing user data, scan results, and reports. FastAPI provides RESTful APIs to connect the frontend, database, and external modules. The Mistral model is integrated via secure API calls to deliver AI-driven vulnerability analysis and recommendations.

2.3.4 Communications Interfaces: The system communicates over TCP/IP using HTTP/HTTPS for API requests. FastAPI handles API communication, while MongoDB exchanges data with the backend. The Mistral model is accessed via secure API calls. Email notifications (e.g., password resets, reports) use SMTP.

System Features

2.4.1 User Authentication & Account Management

Description:
Provides secure login, signup, and role-based access control (e.g., admin, user). Includes password reset and session handling.

Use Case:

- Actors: User, System
- Precondition: User has access to the application.
- Steps:

    1. User enters credentials (email/username + password).
    2. The system verifies details from MongoDB.
    3. If valid, a session token is generated and access is granted.

If invalid, the system prompts error.

4. Users may choose “Forgot Password” to reset via email (SMTP).

- Postcondition: User is authenticated and redirected to dashboard.

2.4.2 Vulnerability Scanning

Description:
Performs security scans on specified targets (e.g., web apps, networks, devices) using integrated tools and custom scripts.

Use Case:

- Actors: User, System

- Precondition: User selects target (IP, domain, or system).
- Steps:
    1. User inputs target details in the scan form.
    2. Backend initiates scan using security modules (e.g., Nmap).
    3. Scan results are collected, processed, and stored in MongoDB.
    4. Results are displayed in the dashboard.
- Postcondition: Vulnerabilities are detected and logged for reporting.

2.4.3 AI Assistant (Mistral Model Integration)

Description:
Provides intelligent explanations and remediation guidance for identified vulnerabilities.

Use Case:

- Actors: User, Mistral Model, System
- Precondition: A vulnerability scan has been completed.
- Steps:
    1. User queries Mistral model.
    2. AI assistant explains the vulnerability, risk level, and mitigation steps.
    3. Response is displayed in a chat-like interface.
- Postcondition: User receives AI-driven assistance for remediation.

2.4.4 Report Generation

Description:
Generates detailed reports of vulnerabilities with severity levels, recommendations, and AI insights. Reports can be exported (PDF/CSV).

Use Case:

- Actors: User, System
- Precondition: At least one completed scan is available.
- Steps:
    1. User selects “Generate Report.”
    2. System compiles scan results.
    3. Report is generated in PDF/CSV format.
    4. User can download or email the report.
- Postcondition: User has a shareable vulnerability report.

2.4.5 Dashboard & Visualization

Description:
Provides a central interface showing scan history, active scans, vulnerability trends, and AI recommendations.

Use Case:

- Actors: User, System
- Precondition: User is logged in.
- Steps:

1. User opens dashboard.
2. System fetches data from MongoDB.
3. Graphs, charts, and summaries are displayed.
4. User can filter by date, severity, or system.

- Postcondition**:** User gains insights into system security posture.

Other Nonfunctional Requirements

2.5.1 Performance Requirements: The system should handle multiple scans concurrently without significant delay. Reports should be generated within a few seconds after scan completion. API response times must remain under 2 seconds for standard queries.

2.5.2 Safety Requirements: The system must not crash target systems during scanning. It should prevent misuse by restricting unauthorized access to scan tools. Sensitive user data must be backed up regularly to avoid data loss.

2.5.3 Security Requirements: All user data, including credentials and scan results, must be encrypted in transit (HTTPS) and at rest (MongoDB encryption). Only authorized users can perform scans. Passwords should be stored using secure hashing. Role-based access must be enforced.

2.5.4 Software Quality Attributes:

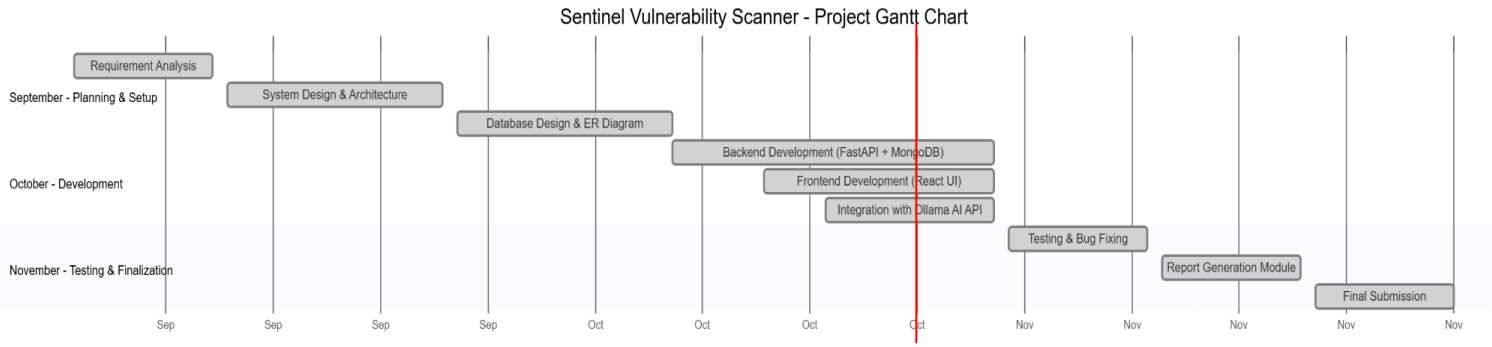
- Reliability: Should maintain consistent performance during long scans.
- Usability: Intuitive dashboard and AI assistant for easy navigation.
- Scalability: Must support growing numbers of users and scans.
- Maintainability: Built using modular architecture (FastAPI + MongoDB).

2.5.5 Business Rules:

- Only registered and verified users can run vulnerability scans.
- Critical vulnerabilities must be flagged and notified immediately.
- Reports must follow a standardized format for professional use.

Chapter 3: Project Management

Gantt Chart



Risk Management & Analysis

3.2.1. Technical Risks:-

3.2.1.1 Compatibility Issues

- **Risk:** The scanner may face compatibility problems across different environments (Windows, Linux, Virtual Machines).
- **Impact:** Medium
- **Likelihood:** Medium
- **Mitigation:**
    - Use cross-platform tools (Python, FastAPI, Nmap).
    - Test the system on multiple OS environments.

3.2.1.2 Nmap Integration Failure

- **Risk:** Nmap may not execute due to missing installation, incorrect PATH, or insufficient privileges.
- **Impact:** High
- **Likelihood:** High
- **Mitigation:**
    - Provide installation checks and error prompts.
    - Use fallback scanning methods.
    - Include documentation for configuring Nmap on all systems.

3.2.1.3 API/Backend Errors

- **Risk:** FastAPI backend may fail due to invalid inputs or unexpected exceptions.
- **Impact:** Medium
- **Likelihood:** Medium
- **Mitigation:**
    - Input validation for IP, ports, ranges.
    - Try/except blocks and meaningful error messages.
    - Logging to monitor backend failures.

3.2.2 Operational Risks

3.2.2.1 Invalid User Input

- **Risk:** A user may enter incorrect IP addresses or unsupported formats.
- **Impact:** Medium
- **Likelihood:** High
- **Mitigation:**
    - Strong frontend validation.
    - Backend validation with informative error responses.

3.2.2.2 Misuse of the Scanner

- **Risk:** Scanning unauthorized or external networks leading to legal or ethical issues.
- **Impact:** High
- **Likelihood:** Low
- **Mitigation:**
    - Display a disclaimer before every scan.
    - Log user scan activity for accountability.

3.2.3 Security Risks

3.2.3.1 Unauthorized Access to Reports

- **Risk:** Unauthenticated users may view confidential or sensitive scan results.
- **Impact:** High
- **Likelihood:** Medium
- **Mitigation:**
    - Enforce JWT-based authentication.
    - Role-based access control (Security Analysts, Managers).
    - Secure storage of reports.

3.2.3.2 Exposure of Sensitive Network Data

- **Risk:** Scan results reveal internal network structure, open ports, and services.
- **Impact:** High
- **Likelihood:** Medium
- **Mitigation:**
    - Encrypt stored reports and logs.
    - Enable protected downloads.

3.2.3.3 Vulnerabilities in the Application Itself

- **Risk:** The scanner web application could have vulnerabilities (XSS, CSRF, SQL Injection).
- **Impact:** High
- **Likelihood:** Medium
- **Mitigation:**
    - Implement secure coding practices.
    - Sanitize API inputs.
    - Conduct internal application testing.

3.2.4 Performance Risks

3.2.4.1 Slow Scan Execution

- **Risk:** Scanning large networks may take a long time.
- **Impact:** Medium
- **Likelihood:** Medium
- **Mitigation:**
    - Allow users to choose between quick scan and full scan.
    - Use parallel or asynchronous scan execution.

3.2.4.2 Backend Overload

- **Risk:** Multiple users scanning simultaneously may overload the server.
- **Impact:** Medium
- **Likelihood:** Low
- **Mitigation:**
    - Limit number of simultaneous scans.
    - Use job queues for heavy tasks.

3.2.5 Project Management Risks

3.2.5.1 Time Constraints

- **Risk:** Development deadlines may not be met due to academic commitments.
- **Impact:** Medium
- **Likelihood:** Medium
- **Mitigation:**
    - Use modular development.
    - Weekly milestones.
    - Task division between team members.

3.2.5.2 Skill Gaps

- **Risk:** Limited experience in network scanning or AI integration may delay implementation.
- **Impact:** Medium
- **Likelihood:** Medium
- **Mitigation:**
    - Research and self-learning.
    - Follow official documentation.
    - Seek faculty support when needed.

3.2.6. Summary

Effective risk management ensures that the Sentinel Vulnerability Scanner remains stable, secure, and reliable. By identifying potential risks early and applying mitigation strategies, the project can be completed successfully within time and quality constraints. Continuous monitoring of risks throughout development and deployment strengthens the system's resilience.

Function Point Analysis

3.3.1 Component Count

Component Type	Description	Count	Complexity Weight
External Inputs (EI)	User inputs for login, scan setup, and sending a prompt to the AI module.	3	4 (Low)
External Outputs (EO)	Reports and vulnerability summaries generated by the system and AI's responses to user prompts.	2	7 (High)
External Inquiries (EQ)	Requests for scan history or results Follow-up prompt / clarification question.	2	5 (Average)
Internal Logical Files (ILF)	Database tables for users, scans, and vulnerabilities.	3	10 (Average)
External Interface Files (EIF)	Integration with Ollama API and threat databases (CVE).	2	5 (Low)

3.3.2 Complexity Factor

Complexity Factor	Rating (0–5)
Backup and recovery	3
Data communications	3
Distributed processing	2
Performance critical	4
Existing operating environment	2
Online data entry	3
Input transaction over multiple screens	1
Master files updated online	3
Information domain values complex	4
Internal processing complex	4
Code designed for reuse	2
Conversion/installation in design	2
Multiple installations	1
Application designed for change	3

Σ Fi = 37

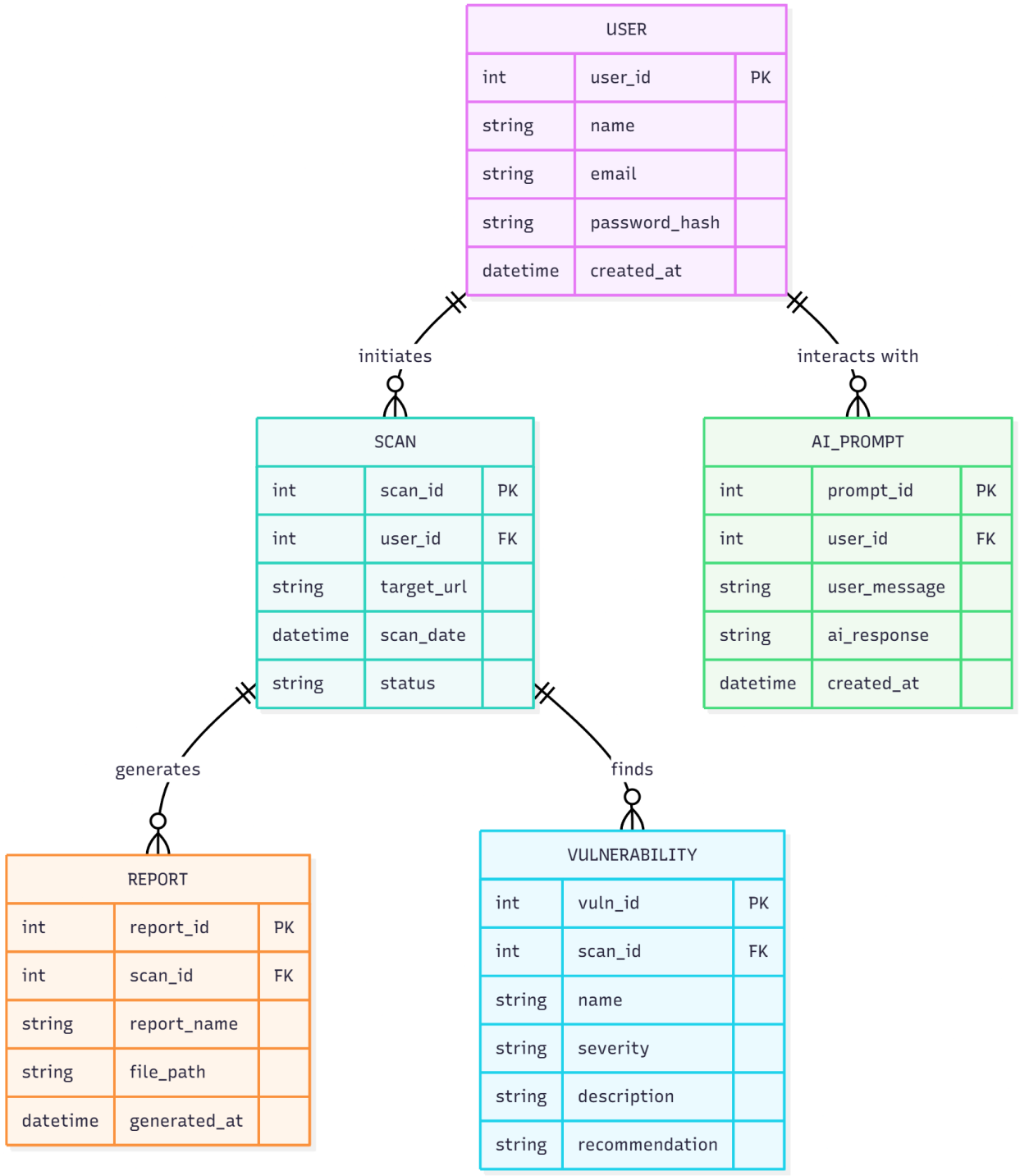
3.3.3 Function Points (FP)

UFP = (3×4) + (2×7) + (2×5) + (3×10) + (2×5) = 12 + 14 + 10 + 30 + 10 = 76
Total complexity factor (Σ Fi) = 37 ⇒ VAF = 0.65 + (37 × 0.01) = 1.02

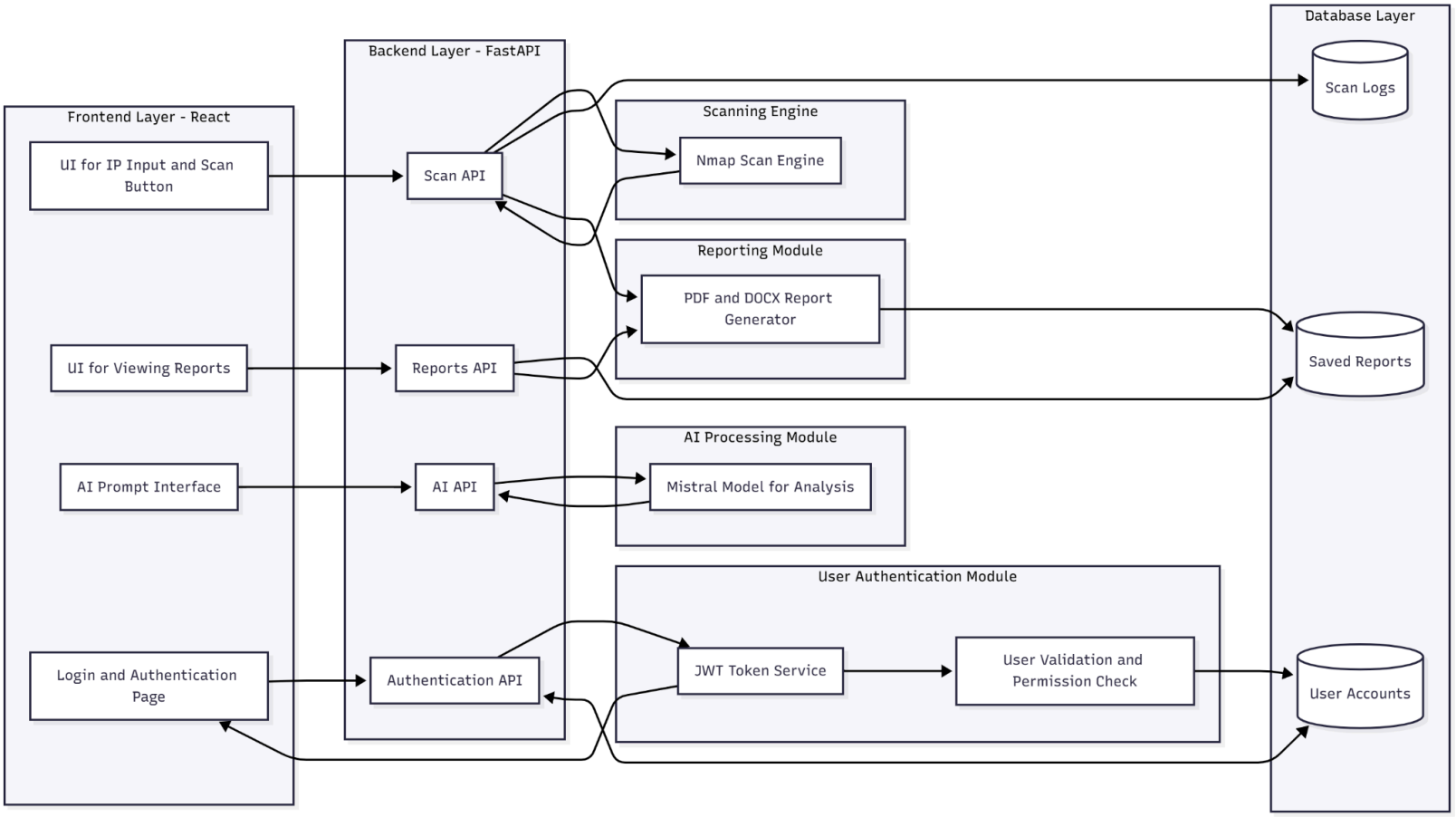
FP = **UFP** × **VAF** = **76** × **1.02** = **77.52 ≈ 78**

Chapter 4: Design & Implementation

ER Diagram & Database Schema



Architectural Design



Implementation (Coding)

Frontend:-

[Frontend]

Open the above link for frontend components

Backend:-

Backend

Open the above link for backend routes

Chapter 5: Testing

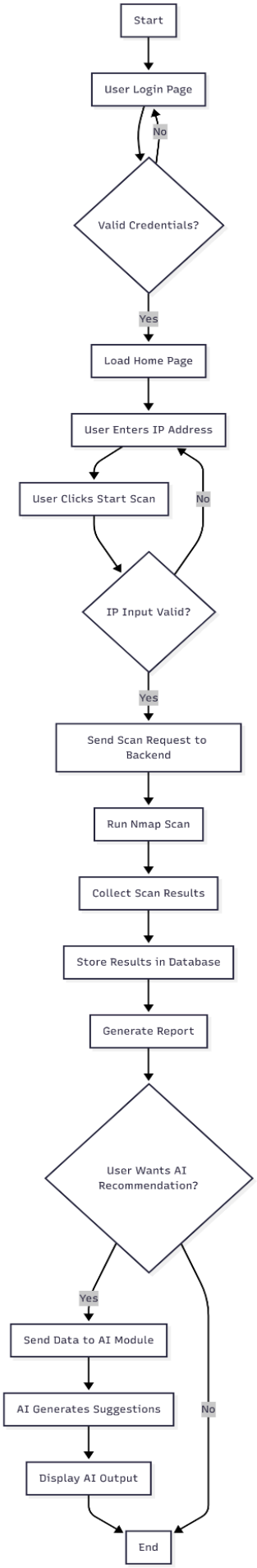
Test Cases:-

TC ID	Test Scenario	Description	Input Data / Preconditions	Expected Result
TC-001	Signup – Valid Data	Verify user can sign up successfully with valid data	Username: testuser, Email: test@mail.com, Password: Test@1234, Confirm Password: Test@1234	User account created successfully and redirected to login page
TC-002	Signup - Existing Email	Verify error when signing up with an already registered email	Email: test@mail.com	System shows “Email already exists”
TC-003	Signup -Password Mismatch	Verify system validates confirm password field	Password: Test@1234, Confirm Password: Test@123	Display “Passwords do not match”
TC-004	Signup - Missing Fields	Verify validation when any required field is empty	Email left blank	Display “Email is required”
TC-005	Signup – Invalid Email Format	Verify validation for improper email input	Email: test@.com	Display “Invalid email format”
TC-006	Signup – Long Input	Verify system limits overly long usernames/emails	Username > 255 chars	Display validation error
TC-007	Login – Valid Credentials	Verify login with correct email and password	Email: test@mail.com, Password: Test@1234	Redirect to dashboard

TC-008	Login – Invalid Password	Verify login failure with incorrect password	Password: wrongpass	Display “Invalid credentials”
TC-009	Login -Unregistered Email	Verify login fails for non-existent user	Email: unknown@mail.com	Display “User not found”
TC-010	Login – Empty Fields	Verify validation for blank input fields	No email/password entered	Display “All fields are required”
TC-011	Login – Case Sensitivity	Verify that password field is case sensitive	Password: test@1234 instead of Test@1234	Login fails
TC-012	Login – Session Handling	Verify user session is created on successful login	Logged in with valid credentials	Session token/cookie created
TC-013	Forgot Password - Valid Email	Verify forgot password process with a registered email	Email: test@mail.com	Reset link sent to user’s email
TC-014	Forgot Password - Unregistered Email	Verify forgot password with unregistered email	Email: notexist@mail.com	Show “Email not registered”
TC-015	Forgot Password - Invalid Format	Verify validation on invalid email	Email: user@.com	Display “Invalid email format”
TC-016	Forgot Password - Empty Field	Verify validation when email field is empty	Email: blank	Display “Email is required”
TC-017	Reset Password - Valid Token	Verify password reset with a valid token	Valid link from email	Password reset successfully

TC-018	Reset Password - Expired Token	Verify expired token handling	Expired reset link	Display “Link expired, please request again”
TC-019	Logout - Valid Session	Verify logout functionality for logged-in user	Active session	Session ends, redirect to login
TC-020	Logout - Without Session	Verify logout handling when no active session	No login	Redirect to login page
TC-021	Authentication - Access Control	Verify that unauthenticated users can’t access protected routes	Try accessing /dashboard without login	Redirect to login
TC-022	Password Storage Security	Verify that passwords are hashed in the database	Check DB entries	Password stored in hash format (not plaintext)
TC-023	Error Message Clarity	Verify system does not leak sensitive info in errors	Enter wrong credentials	Generic message: “Invalid credentials”

Flow Graph:-


Start
User Login Page
No
Valid Credentials?
Yes
Load Home Page
User Enters IP Address
User Clicks Start Scan
No
IP Input Valid?
Yes
Send Scan Request to Backend
Run Nmap Scan
Collect Scan Results
Store Results in Database
Generate Report
User Wants AI Recommendation?
Yes
Send Data to AI Module
AI Generates Suggestions
No
Display AI Output
End

Cyclomatic Complexity (V(G))

Cyclomatic Complexity Formula (Standard)

$V(G)=D+1$

Where:

- D = number of decision nodes
- D = 4

So:

$V(G)=3+1=4$

References

Books

1. Sommerville, Ian. *Software Engineering*, 10th Edition, Pearson Education.

2. Pressman, Roger S., and Maxim, Bruce R. *Software Engineering: A Practitioner's Approach*, McGraw-Hill Education.

3. Stallings, William. *Network Security Essentials: Applications and Standards*, Pearson.

4. Allen, Julia H. *The CERT Guide to System and Network Security Practices*, Addison-Wesley.

Research Papers / Standards

5. ISO/IEC 27001: Information Security Management Systems.

6. CVE® Program Documentation, MITRE Corporation.

7. OWASP (2023). *OWASP Testing Guide*. OWASP Foundation.

Websites

8. Nmap Network Scanning: Official Documentation. https://nmap.org/book/man.html

9. FastAPI Documentation. https://fastapi.tiangolo.com/

10. React.js Official Documentation.
https://react.dev/

11. Python-Nmap Library Documentation.
https://xael.org/pages/python-nmap-en.html

12. Docker DVWA Project Documentation.
https://github.com/digininja/DVWA

13. Mistral AI Official API Documentation.
https://docs.mistral.ai/

14. MDN Web Docs – JavaScript Fetch API.
https://developer.mozilla.org/

15. JSON Formatter – JSON Output Understanding.
https://jsonformatter.org/

16. JWT (JSON Web Token) Documentation.
https://jwt.io/

Software and Tools Used

17. Visual Studio Code – Code Editor.

18. Postman – API Testing Tool.

19. Docker Engine & Docker Desktop/CLI.

20. VMware Workstation Pro – Virtualization.

21. Parrot Security OS – Testing Environment.

22. Nmap & Zenmap – Network Scanning.

23. Git & GitHub – Version Control.

24. Google Docs – Report Preparation.

Online Tools

25. Mermaid Live Editor – Diagram Generation.
    https://mermaid.live

AI & LLM Tools

28. Mistral AI – Model used for intelligent recommendations.

29. ChatGPT / GPT-5.1 – Used for documentation support & content refinement.